

In [37]:

```

1 import pandas as pd
2
3 aa = pd.read_csv("matches.csv")
4
5 aa.head()
6

```

Out[37]:

	id	season	city	date	match_type	player_of_match	venue	team1	team2	toss_winner
0	335982	2007/08	Bangalore	2008-04-18	League	BB McCullum	M Chinnaswamy Stadium	Royal Challengers Bangalore	Kolkata Knight Riders	Roy Challenge Bangalo
1	335983	2007/08	Chandigarh	2008-04-19	League	MEK Hussey	Punjab Cricket Association Stadium, Mohali	Kings XI Punjab	Chennai Super Kings	Chenn Super King
2	335984	2007/08	Delhi	2008-04-19	League	MF Maharoo	Feroz Shah Kotla	Delhi Daredevils	Rajasthan Royals	Rajastha Roya
3	335985	2007/08	Mumbai	2008-04-20	League	MV Boucher	Wankhede Stadium	Mumbai Indians	Royal Challengers Bangalore	Mumb Indiar
4	335986	2007/08	Kolkata	2008-04-20	League	DJ Hussey	Eden Gardens	Kolkata Knight Riders	Deccan Chargers	Decca Charge

```
In [39]: 1 print("Rows and Columns:", aa.shape)
2 aa.info()
3 # Check for missing values
4 print("\nMissing Values:\n", aa.isnull().sum())
5 # Get unique columns
6 print("\nColumns:\n", aa.columns.tolist())
7
```

```
Rows and Columns: (1095, 20)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1095 entries, 0 to 1094
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     1095 non-null   int64
1   season                 1095 non-null   object
2   city                   1044 non-null   object
3   date                   1095 non-null   object
4   match_type             1095 non-null   object
5   player_of_match        1090 non-null   object
6   venue                  1095 non-null   object
7   team1                  1095 non-null   object
8   team2                  1095 non-null   object
9   toss_winner            1095 non-null   object
10  toss_decision          1095 non-null   object
11  winner                 1090 non-null   object
12  result                 1095 non-null   object
13  result_margin          1076 non-null   float64
14  target_runs            1092 non-null   float64
15  target_overs           1092 non-null   float64
16  super_over             1095 non-null   object
17  method                 21 non-null     object
18  umpire1                1095 non-null   object
19  umpire2                1095 non-null   object
dtypes: float64(3), int64(1), object(16)
memory usage: 171.2+ KB
```

```
Missing Values:
id                0
season            0
city              51
date              0
match_type        0
player_of_match   5
venue             0
team1             0
team2             0
toss_winner       0
toss_decision     0
winner            5
result            0
result_margin     19
target_runs       3
target_overs      3
super_over        0
method            1074
umpire1           0
umpire2           0
dtype: int64
```

```
Columns:
['id', 'season', 'city', 'date', 'match_type', 'player_of_match', 'venue', 'team1', 'team2',
'toss_winner', 'toss_decision', 'winner', 'result', 'result_margin', 'target_runs', 'target_overs',
'super_over', 'method', 'umpire1', 'umpire2']
```

```
In [40]: 1 # List of selected features
2 features = ['team1', 'team2', 'toss_winner', 'toss_decision', 'city', 'venue', 'season']
3 target = 'winner'
4
5 # Check unique values
6 for col in features + [target]:
7     print(f"\n{col}:\n", aa[col].value_counts())
8
9 # Check missing values
10 aa[features + [target]].isnull().sum()
11
```

Dubai	13
Dharamsala	13
Centurion	12
Rajkot	10
Sharjah	10
Indore	9
Navi Mumbai	9
Johannesburg	8
Cuttack	7
Port Elizabeth	7
Ranchi	7
Cape Town	7
Raipur	6
Mohali	5
Kochi	5
Kanpur	4
Kimberley	3
East London	3
Guwahati	3
Nagpur	3

```
In [41]: 1 # Drop rows where any of the key columns are missing
2 aa_cleaned = aa.dropna(subset=features + [target])
```

```
In [42]: 1 from sklearn.preprocessing import LabelEncoder
2
3 aa_encoded = aa_cleaned.copy()
4
5 encoder = LabelEncoder()
6 for col in features + [target]:
7     aa_encoded[col] = encoder.fit_transform(aa_encoded[col])
8
```

In [43]:

```
1 aa_cleaned
```

Out[43]:

	id	season	city	date	match_type	player_of_match	venue	team1	team2	toss_
0	335982	2007/08	Bangalore	2008-04-18	League	BB McCullum	M Chinnaswamy Stadium	Royal Challengers Bangalore	Kolkata Knight Riders	Chal Ba
1	335983	2007/08	Chandigarh	2008-04-19	League	MEK Hussey	Punjab Cricket Association Stadium, Mohali	Kings XI Punjab	Chennai Super Kings	C Supe
2	335984	2007/08	Delhi	2008-04-19	League	MF Maharoo	Feroz Shah Kotla	Delhi Daredevils	Rajasthan Royals	Ra
3	335985	2007/08	Mumbai	2008-04-20	League	MV Boucher	Wankhede Stadium	Mumbai Indians	Royal Challengers Bangalore	M
4	335986	2007/08	Kolkata	2008-04-20	League	DJ Hussey	Eden Gardens	Kolkata Knight Riders	Deccan Chargers	I Cl
...	...	...	...	...	...	...	...	...	...	...
1090	1426307	2024	Hyderabad	2024-05-19	League	Abhishek Sharma	Rajiv Gandhi International Stadium, Uppal, Hyd...	Punjab Kings	Sunrisers Hyderabad	
1091	1426309	2024	Ahmedabad	2024-05-21	Qualifier 1	MA Starc	Narendra Modi Stadium, Ahmedabad	Sunrisers Hyderabad	Kolkata Knight Riders	St Hyd
1092	1426310	2024	Ahmedabad	2024-05-22	Eliminator	R Ashwin	Narendra Modi Stadium, Ahmedabad	Royal Challengers Bengaluru	Rajasthan Royals	Ra
1093	1426311	2024	Chennai	2024-05-24	Qualifier 2	Shahbaz Ahmed	MA Chidambaram Stadium, Chepauk, Chennai	Sunrisers Hyderabad	Rajasthan Royals	Ra
1094	1426312	2024	Chennai	2024-05-26	Final	MA Starc	MA Chidambaram Stadium, Chepauk, Chennai	Sunrisers Hyderabad	Kolkata Knight Riders	St Hyd

1039 rows × 20 columns



In [85]:

```
1 aa
2
```

Out[85]:

	id	season	city	date	match_type	player_of_match	venue	team1	team2	toss_
0	335982	2007/08	Bangalore	2008-04-18	League	BB McCullum	M Chinnaswamy Stadium	Royal Challengers Bangalore	Kolkata Knight Riders	Chal Ba
1	335983	2007/08	Chandigarh	2008-04-19	League	MEK Hussey	Punjab Cricket Association Stadium, Mohali	Kings XI Punjab	Chennai Super Kings	C Supe
2	335984	2007/08	Delhi	2008-04-19	League	MF Maharoof	Feroz Shah Kotla	Delhi Daredevils	Rajasthan Royals	Ra
3	335985	2007/08	Mumbai	2008-04-20	League	MV Boucher	Wankhede Stadium	Mumbai Indians	Royal Challengers Bangalore	↑
4	335986	2007/08	Kolkata	2008-04-20	League	DJ Hussey	Eden Gardens	Kolkata Knight Riders	Deccan Chargers	I Cl
...	...	...	...	...	...	...	...	...	...	...
1090	1426307	2024	Hyderabad	2024-05-19	League	Abhishek Sharma	Rajiv Gandhi International Stadium, Uppal, Hyd...	Punjab Kings	Sunrisers Hyderabad	
1091	1426309	2024	Ahmedabad	2024-05-21	Qualifier 1	MA Starc	Narendra Modi Stadium, Ahmedabad	Sunrisers Hyderabad	Kolkata Knight Riders	St Hyd
1092	1426310	2024	Ahmedabad	2024-05-22	Eliminator	R Ashwin	Narendra Modi Stadium, Ahmedabad	Royal Challengers Bengaluru	Rajasthan Royals	Ra
1093	1426311	2024	Chennai	2024-05-24	Qualifier 2	Shahbaz Ahmed	MA Chidambaram Stadium, Chepauk, Chennai	Sunrisers Hyderabad	Rajasthan Royals	Ra
1094	1426312	2024	Chennai	2024-05-26	Final	MA Starc	MA Chidambaram Stadium, Chepauk, Chennai	Sunrisers Hyderabad	Kolkata Knight Riders	St Hyd

1095 rows × 20 columns

In [44]:

```
1
2 columns_to_keep = [
3     'season', 'city', 'venue', 'match_type', 'team1', 'team2',
4     'toss_winner', 'toss_decision', 'super_over',
5     'winner'
6 ]
7 aa_model = aa_encoded[columns_to_keep]
8 aa_model.head()
9
```

Out[44]:

	season	city	venue	match_type	team1	team2	toss_winner	toss_decision	super_over	winner
0	0	2	23	League	16	8	16	1	N	8
1	0	7	40	League	6	0	0	0	N	0
2	0	10	16	League	3	13	13	0	N	3
3	0	26	55	League	10	16	10	0	N	16
4	0	23	14	League	8	1	1	0	N	8

In [45]: 1 aa_model

Out[45]:

	season	city	venue	match_type	team1	team2	toss_winner	toss_decision	super_over	winner
0	0	2	23	League	16	8	16	1	N	8
1	0	7	40	League	6	0	0	0	N	0
2	0	10	16	League	3	13	13	0	N	3
3	0	26	55	League	10	16	10	0	N	16
4	0	23	14	League	8	1	1	0	N	8
...	...	...	...	...	...	...	...	...	...	...
1090	16	16	43	League	12	18	12	0	N	18
1091	16	1	32	Qualifier 1	18	8	18	0	N	8
1092	16	1	32	Eliminator	17	13	13	1	N	13
1093	16	8	28	Qualifier 2	18	13	13	1	N	18
1094	16	8	28	Final	18	8	18	0	N	8

1039 rows × 10 columns

```

In [46]: 1 from sklearn.preprocessing import LabelEncoder
2
3 # Create a new encoder instance
4 le_match_type = LabelEncoder()
5
6 # Apply encoding to match_type
7 aa_model['match_type'] = le_match_type.fit_transform(aa_model['match_type'])
8
9 # Check the result
10 print("Encoding Mapping for match_type:")
11 for cls, val in zip(le_match_type.classes_, le_match_type.transform(le_match_type.classes_)):
12     print(f"{cls}: {val}")
13
14 a_model.head()
15

```

Encoding Mapping for match_type:

3rd Place Play-Off: 0

Elimination Final: 1

Eliminator: 2

Final: 3

League: 4

Qualifier 1: 5

Qualifier 2: 6

Semi Final: 7

Out[46]:

	season	city	venue	match_type	team1	team2	toss_winner	toss_decision	super_over	winner
0	0	2	23	4	16	8	16	1	NaN	8
1	0	7	40	4	6	0	0	0	NaN	0
2	0	10	16	4	3	13	13	0	NaN	3
3	0	26	55	4	10	16	10	0	NaN	16
4	0	23	14	4	8	1	1	0	NaN	8

```
In [47]: 1 aa_model.isnull().sum()
         2
```

```
Out[47]: season      0
         city        0
         venue       0
         match_type  0
         team1       0
         team2       0
         toss_winner  0
         toss_decision 0
         super_over   0
         winner      0
         dtype: int64
```

```
In [48]: 1 aa_model['super_over'].unique()
         2
```

```
Out[48]: array(['N', 'Y'], dtype=object)
```

```
In [49]: 1 # Strip whitespace and convert to uppercase
         2 aa_model['super_over'] = aa_model['super_over'].astype(str).str.strip().str.upper()
         3
         4 # Map to 1/0
         5 aa_model['super_over'] = aa_model['super_over'].map({'N': 0, 'Y': 1})
         6 import warnings
         7 warnings.filterwarnings('ignore')
```

```
In [50]: 1 aa_model['super_over'].value_counts(dropna=False)
         2
```

```
Out[50]: super_over
         0    1028
         1     11
         Name: count, dtype: int64
```

```
In [51]: 1 # Get summary statistics for numeric columns
         2 aa_model.describe()
         3
```

```
Out[51]:
```

	season	city	venue	match_type	team1	team2	toss_winner	toss_decision	su
count	1039.000000	1039.000000	1039.000000	1039.000000	1039.000000	1039.000000	1039.000000	1039.000000	1039.000000
mean	8.008662	15.938402	30.617902	4.007700	8.687199	8.789220	8.546679	0.644851	0.644851
std	5.009136	9.948909	16.551121	0.466548	5.735612	5.555885	5.638729	0.478789	0.478789
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	4.000000	8.000000	16.000000	4.000000	3.000000	4.000000	3.000000	0.000000	0.000000
50%	8.000000	16.000000	28.000000	4.000000	8.000000	9.000000	8.000000	1.000000	1.000000
75%	13.000000	26.000000	46.000000	4.000000	13.000000	13.000000	13.000000	1.000000	1.000000
max	16.000000	35.000000	57.000000	7.000000	18.000000	18.000000	18.000000	1.000000	1.000000

```
In [53]: 1 print("Mean:\n", aa_model.mean(numeric_only=True))
2 print("\nMedian:\n", aa_model.median(numeric_only=True))
3 print("\nMode:\n", aa_model.mode(numeric_only=True).iloc[0])
4 print("\nStandard Deviation:\n", aa_model.std(numeric_only=True))
5 range_vals = aa_model.max(numeric_only=True) - aa_model.min(numeric_only=True)
6 print("\nRange:\n", range_vals)
7
```

```
Mean:
  season      8.008662
  city      15.938402
  venue      30.617902
  match_type   4.007700
  team1       8.687199
  team2       8.789220
  toss_winner   8.546679
  toss_decision  0.644851
  super_over    0.010587
  winner       8.661213
dtype: float64
```

```
Median:
  season      8.0
  city      16.0
  venue      28.0
  match_type   4.0
  team1       8.0
  team2       9.0
  toss_winner   8.0
  toss_decision  1.0
  super_over    0.0
  winner       8.0
dtype: float64
```

```
Mode:
  season      5
  city      26
  venue      14
  match_type   4
  team1       0
  team2      10
  toss_winner  10
  toss_decision  1
  super_over    0
  winner      10
Name: 0, dtype: int64
```

```
Standard Deviation:
  season      5.009136
  city      9.948909
  venue     16.551121
  match_type   0.466548
  team1       5.735612
  team2       5.555885
  toss_winner   5.638729
  toss_decision  0.478789
  super_over    0.102397
  winner       5.653681
dtype: float64
```

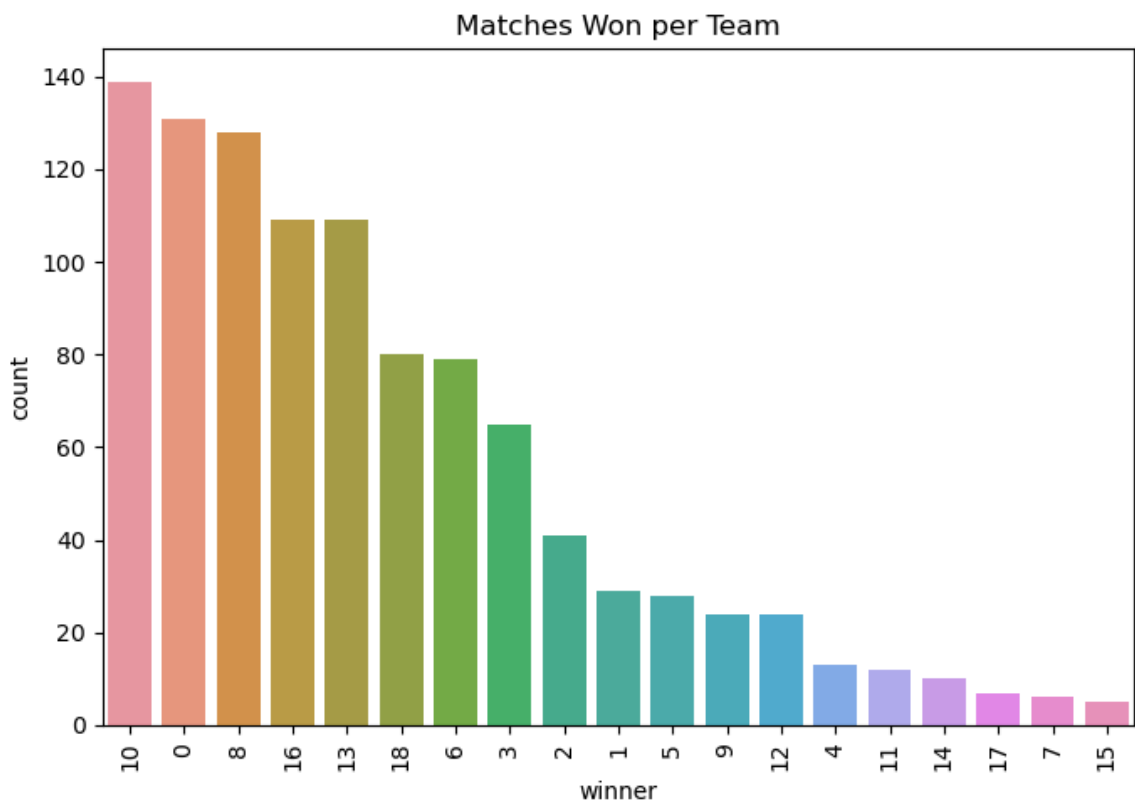
```
Range:
  season      16
  city      35
  venue      57
  match_type   7
  team1      18
  team2      18
  toss_winner  18
  toss_decision  1
  super_over    1
  winner      18
dtype: int64
```



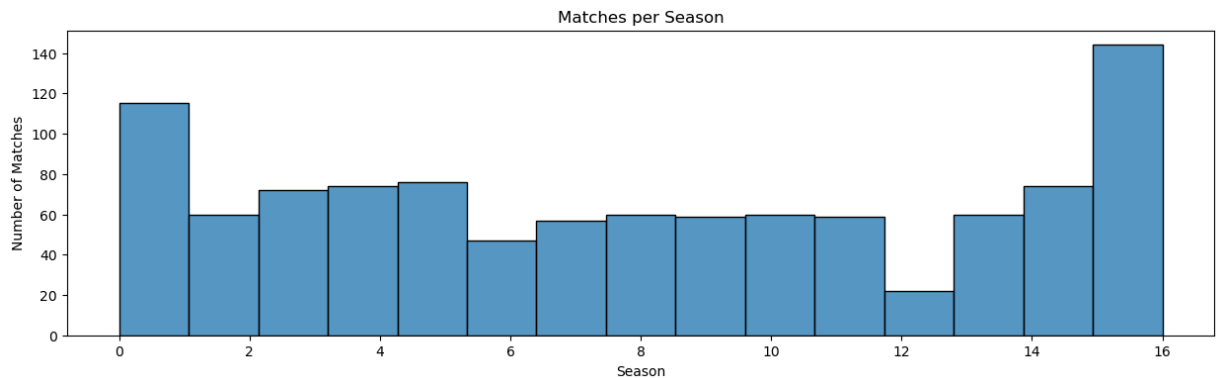
```
In [54]: 1 categorical_cols = ['team1', 'team2', 'toss_winner', 'toss_decision', 'city', 'venue', 'mat
2
3 for col in categorical_cols:
4     print(f"\n{col}:\n", aa_model[col].value_counts())
5
```

```
team1:
team1
0      124
16     123
10     118
8      116
13     96
6      86
3      82
18     80
1      39
2      32
12     31
11     23
9      22
5      21
4      16
17      9
18      -
```

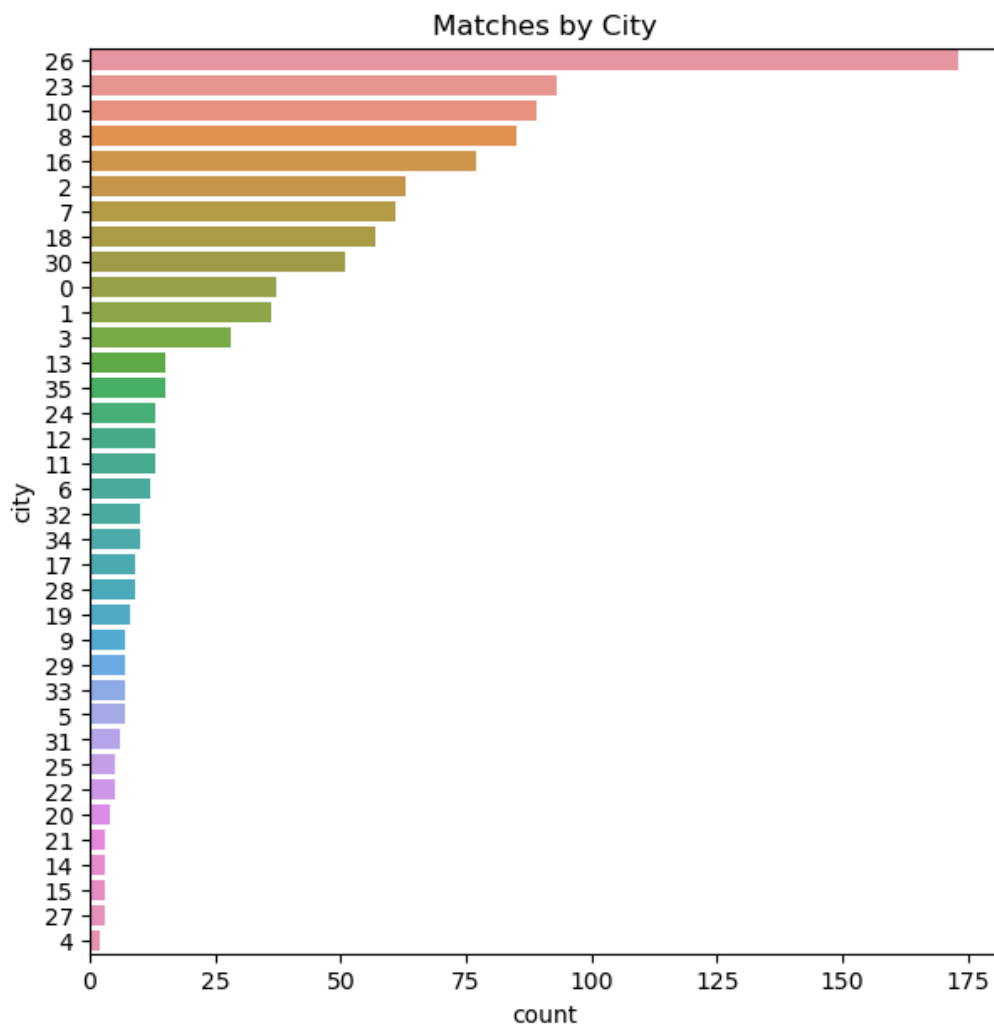
```
In [55]: 1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 plt.figure(figsize=(7, 5))
5 sns.countplot(x='winner', data=aa_model, order=aa_model['winner'].value_counts().index)
6 plt.xticks(rotation=90)
7 plt.title('Matches Won per Team')
8 plt.tight_layout()
9 plt.show()
10
```



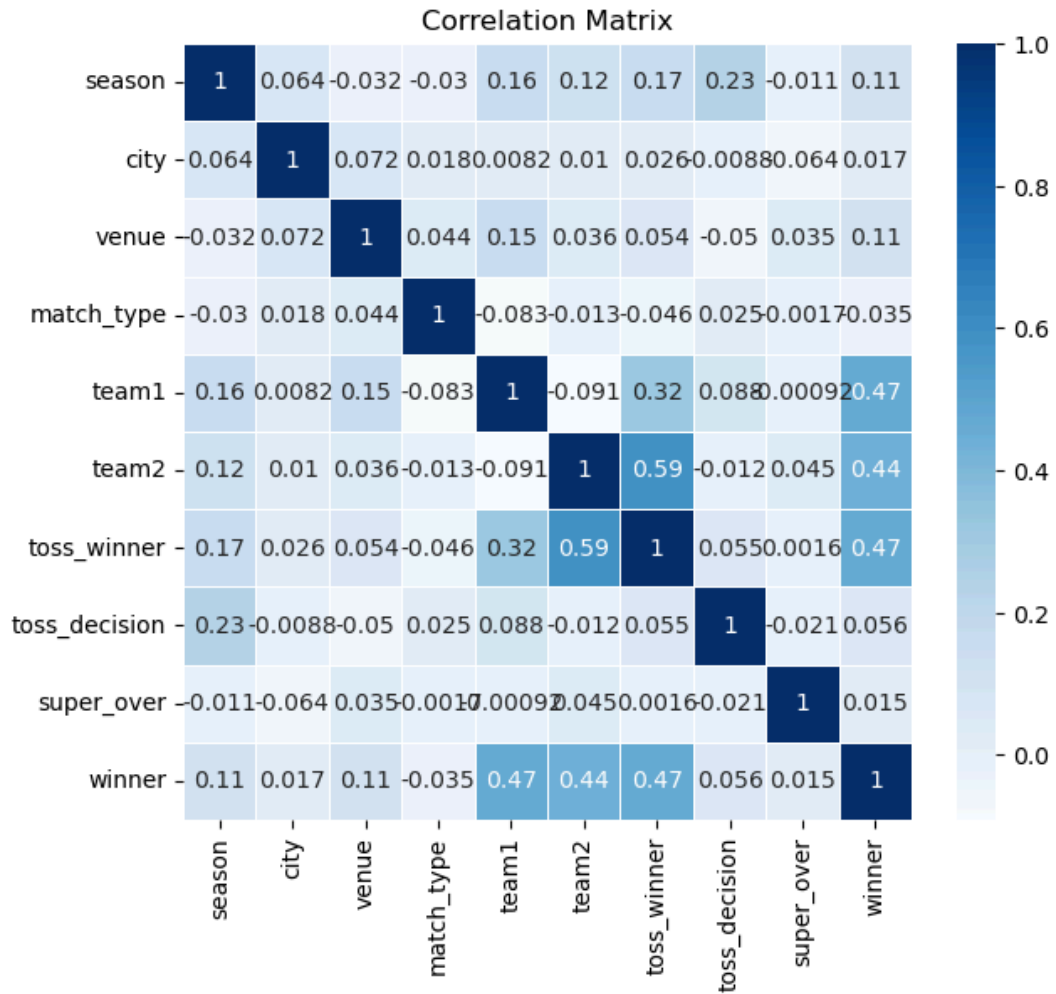
```
In [56]: 1 plt.figure(figsize=(15, 4))
2 sns.histplot(a_model['season'], bins=15, kde=False)
3 plt.title('Matches per Season')
4 plt.xlabel('Season')
5 plt.ylabel('Number of Matches')
6 plt.show()
7
```



```
In [32]: 1 plt.figure(figsize=(7, 7))
2 sns.countplot(y='city', data=a_model, order=a_model['city'].value_counts().index)
3 plt.title('Matches by City')
4 plt.show()
5
```

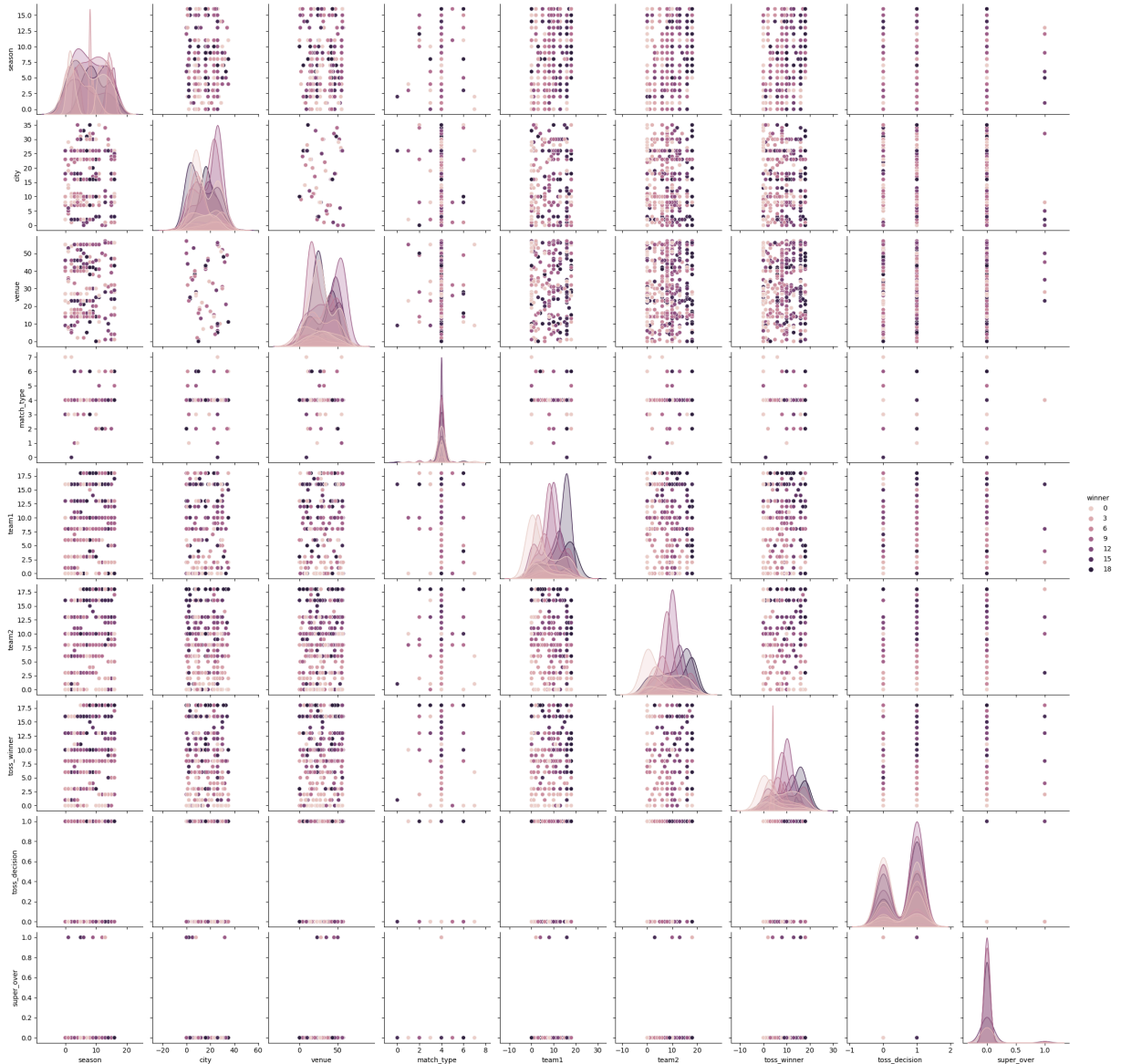


```
In [57]: 1 plt.figure(figsize=(7, 6))
2 sns.heatmap(aa_model.corr(numeric_only=True), annot=True, cmap='Blues', linewidths=0.5)
3 plt.title('Correlation Matrix')
4 plt.show()
5
```



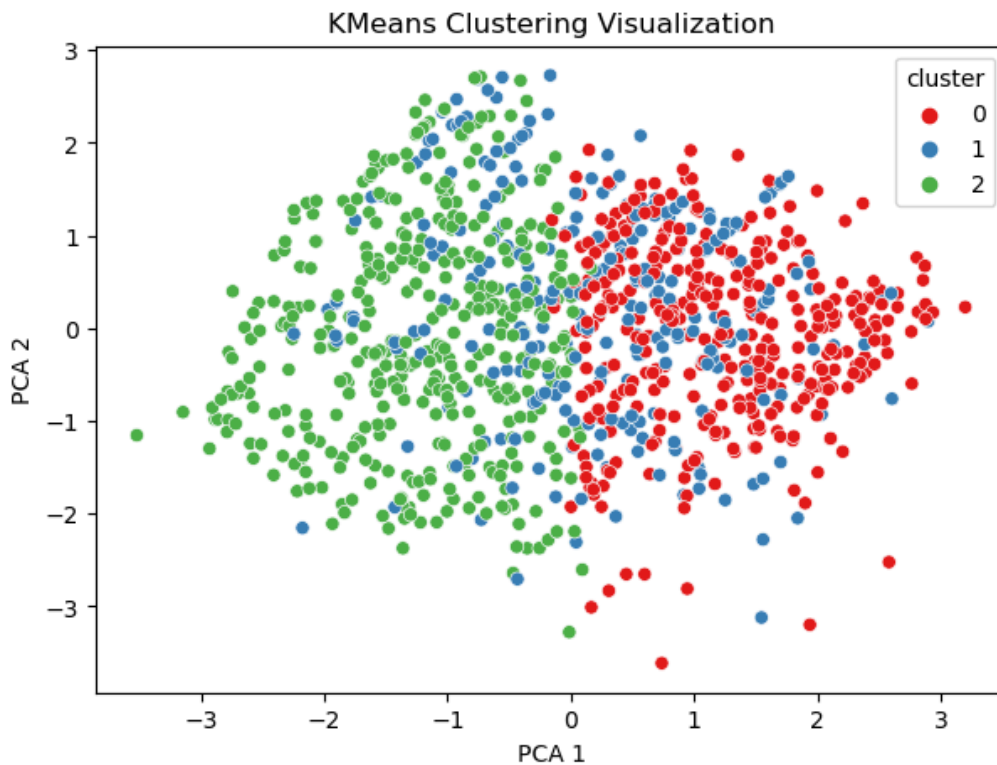
```
In [58]: 1 import seaborn as sns
2
3 # Sample a smaller subset for performance if needed
4 sample_aa = aa_model.sample(n=500, random_state=1)
5
6 sns.pairplot(sample_aa, hue='winner', diag_kind='kde')
7
```

Out[58]: <seaborn.axisgrid.PairGrid at 0x26600d30d50>



```
In [61]: 1 from sklearn.cluster import KMeans
2 from sklearn.preprocessing import StandardScaler
3
4 # Standardize data
5 scaler = StandardScaler()
6 X_scaled = scaler.fit_transform(aa_model.drop(columns=['winner']))
7
8 # Fit KMeans (let's try 3 clusters)
9 kmeans = KMeans(n_clusters=3, random_state=42)
10 aa_model['cluster'] = kmeans.fit_predict(X_scaled)
11
```

```
In [62]: 1 from sklearn.decomposition import PCA
2 import matplotlib.pyplot as plt
3
4 # Reduce to 2 dimensions for plotting
5 pca = PCA(n_components=2)
6 X_pca = pca.fit_transform(X_scaled)
7
8 plt.figure(figsize=(7, 5))
9 sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=aa_model['cluster'], palette='Set1')
10 plt.title('KMeans Clustering Visualization')
11 plt.xlabel('PCA 1')
12 plt.ylabel('PCA 2')
13 plt.show()
14
```



Objective: Analytical Goal To predict the winner of an IPL match based on pre-match information such as teams, venue, toss outcome, and match type. Additionally, to explore which factors most influence the match outcome.

Feature Engineering Created new insights such as whether a super over occurred and encoded categorical variables. Additional derived features like team win consistency or home advantage can be explored further for future enhancement.

```
In [63]: 1 aa_model['toss_win_match_win'] = (aa_model['toss_winner'] == aa_model['winner']).astype(int)
```

Model Selection Used a Decision Tree Classifier for its interpretability and ability to handle categorical data. Since the target (winner) is categorical, classification models are appropriate.

```
In [64]: 1 from sklearn.tree import DecisionTreeClassifier
```

Training and Evaluation The dataset was split into training and testing sets using an 80-20 split. The model was evaluated using Accuracy, F1-Score, Classification Report, and a Confusion Matrix to measure how well the model performed on unseen data.

```
In [65]: 1 from sklearn.model_selection import train_test_split
2
3 X = aa_model.drop(columns=['winner'])
4 y = aa_model['winner']
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.2, random_state=42)
8
```

```
In [66]: 1 model = DecisionTreeClassifier(random_state=42)
2 model.fit(X_train, y_train)
3
4 y_pred = model.predict(X_test)
5
```

```
In [67]: 1 from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 print("Accuracy:", accuracy_score(y_test, y_pred))
6 print("F1 Score (macro):", f1_score(y_test, y_pred, average='macro'))
7
```

Accuracy: 0.5240384615384616
F1 Score (macro): 0.38101795659042603

```
In [68]: 1 print("\nClassification Report:\n", classification_report(y_test, y_pred))
2
```

Classification Report:

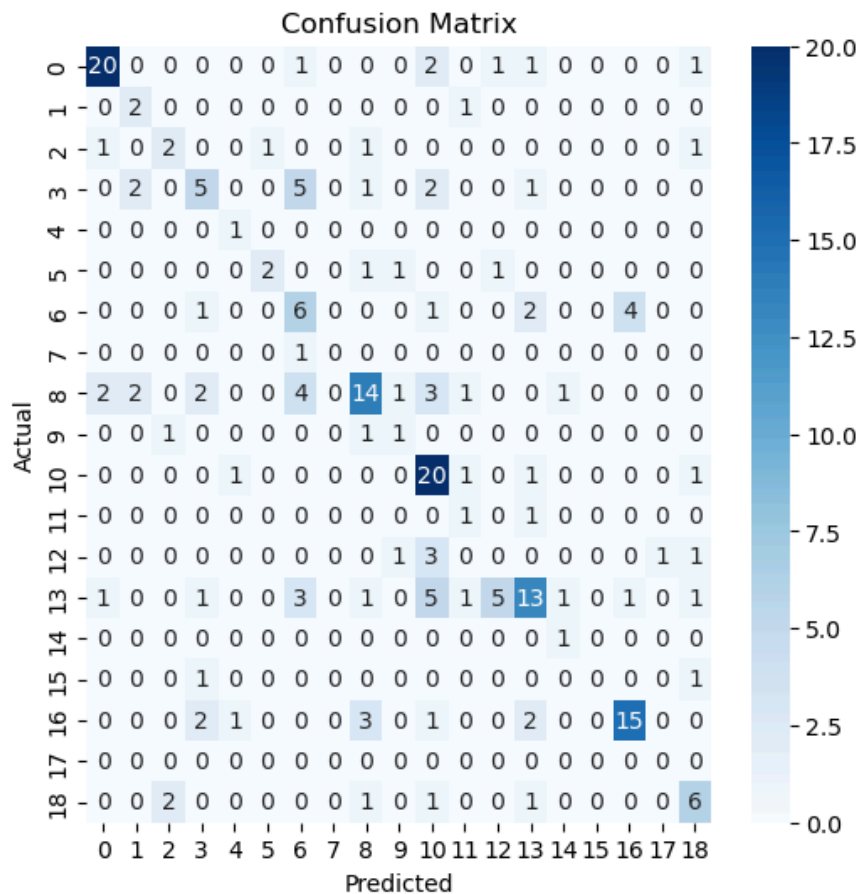
	precision	recall	f1-score	support
0	0.83	0.77	0.80	26
1	0.33	0.67	0.44	3
2	0.40	0.33	0.36	6
3	0.42	0.31	0.36	16
4	0.33	1.00	0.50	1
5	0.67	0.40	0.50	5
6	0.30	0.43	0.35	14
7	0.00	0.00	0.00	1
8	0.61	0.47	0.53	30
9	0.25	0.33	0.29	3
10	0.53	0.83	0.65	24
11	0.20	0.50	0.29	2
12	0.00	0.00	0.00	6
13	0.59	0.39	0.47	33
14	0.33	1.00	0.50	1
15	0.00	0.00	0.00	2
16	0.75	0.62	0.68	24
17	0.00	0.00	0.00	0
18	0.50	0.55	0.52	11
accuracy				0.52
macro avg				0.37
weighted avg				0.55

In [69]:

```

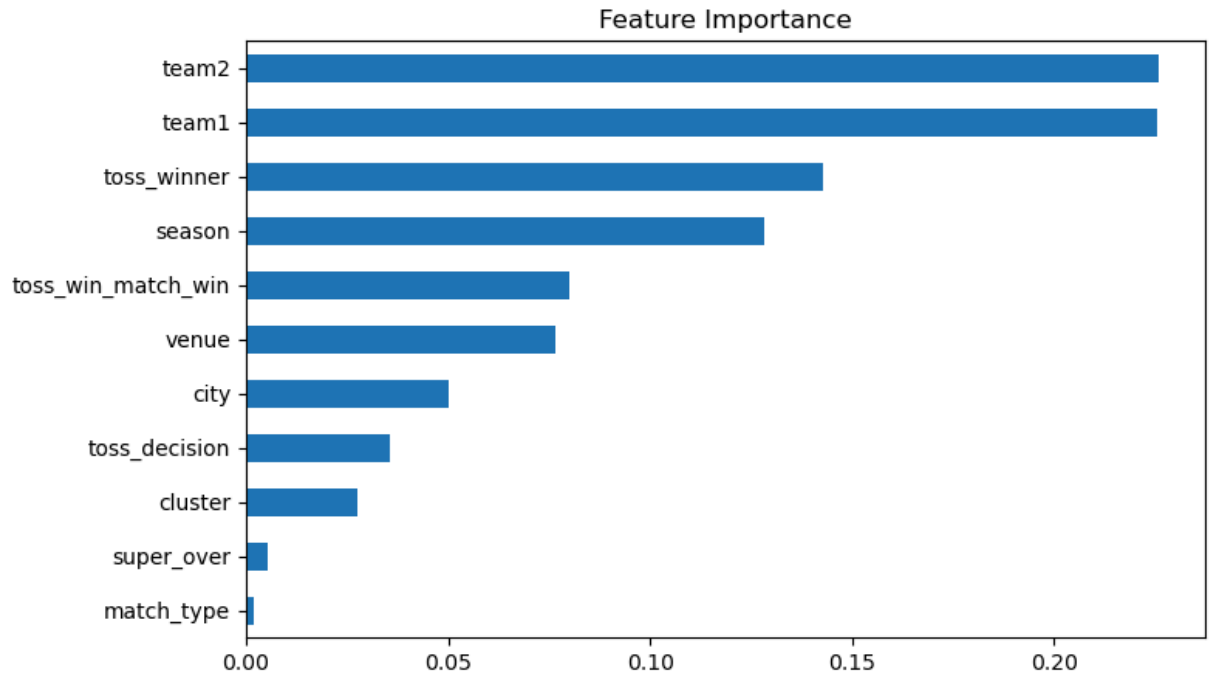
1
2 # Confusion Matrix
3 plt.figure(figsize=(6,6))
4 sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues')
5 plt.title("Confusion Matrix")
6 plt.xlabel("Predicted")
7 plt.ylabel("Actual")
8 plt.show()
9

```



Model Interpretation We analyzed the Decision Tree model's feature importance to identify which pre-match features influenced the prediction most. Toss outcome and venue showed strong influence, suggesting potential strategic impact on results.

```
In [70]: 1 # Feature Importance
2 import pandas as pd
3
4 feat_importance = pd.Series(model.feature_importances_, index=X.columns)
5 feat_importance.sort_values(ascending=True).plot(kind='barh', figsize=(8,5), title="Feature
6 plt.show()
7
```

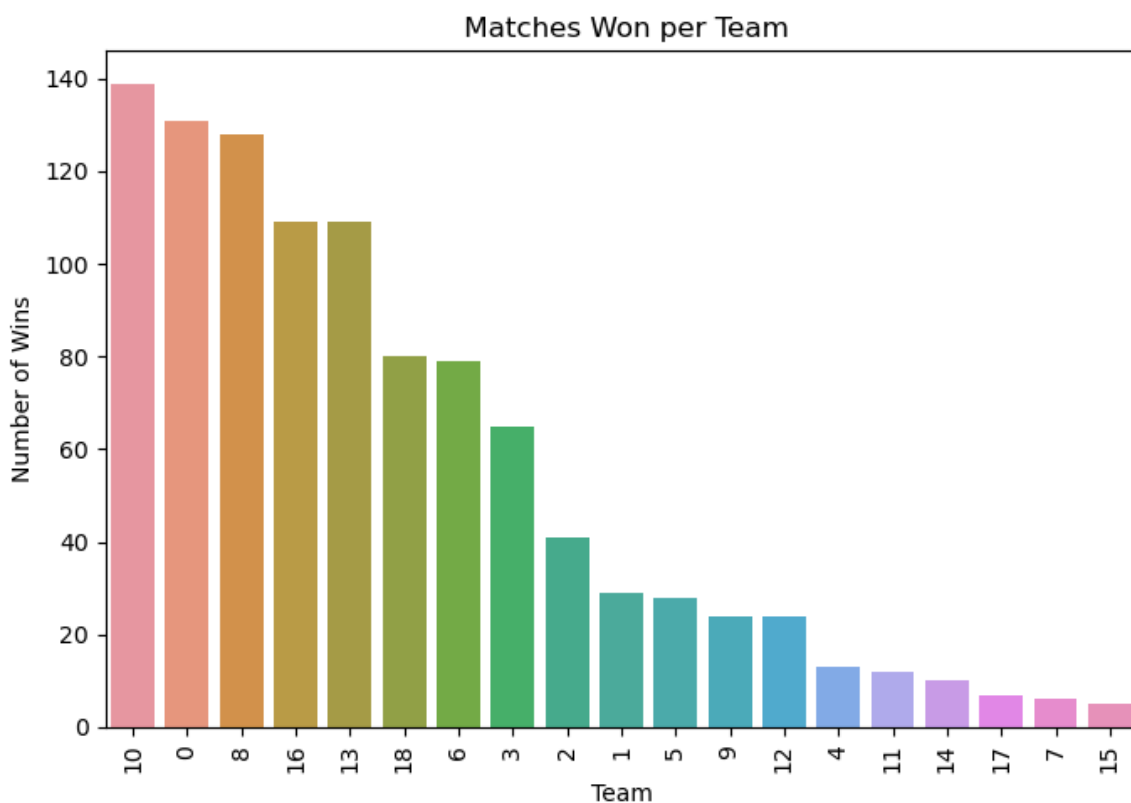


Visualization

Visualization Visualized key aspects of the IPL data to uncover win patterns, toss impacts, and dominant teams. Charts highlight not only historical trends but also how model predictions correlate with real-world strategies.

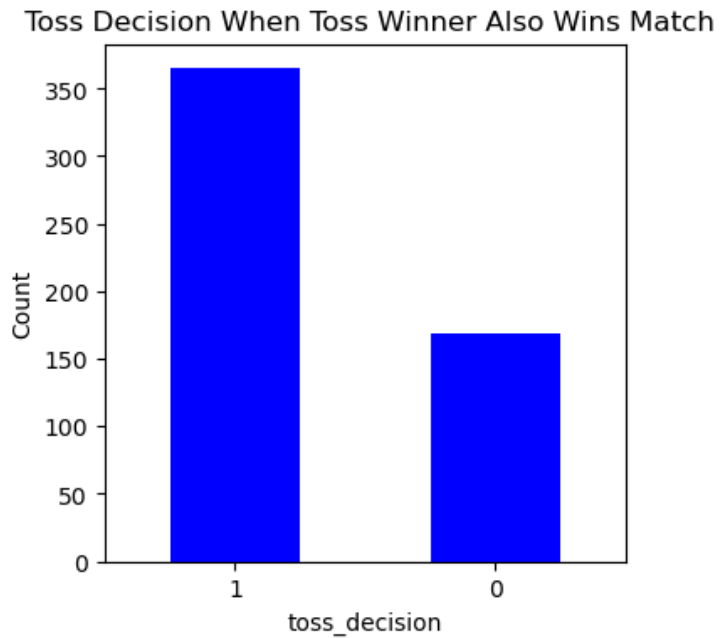
A. Wins per Team (Bar Chart)


```
In [71]: 1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 plt.figure(figsize=(7, 5))
5 sns.countplot(data=aa_model, x='winner', order=aa_model['winner'].value_counts().index)
6 plt.xticks(rotation=90)
7 plt.title(' Matches Won per Team')
8 plt.ylabel('Number of Wins')
9 plt.xlabel('Team')
10 plt.tight_layout()
11 plt.show()
12
```



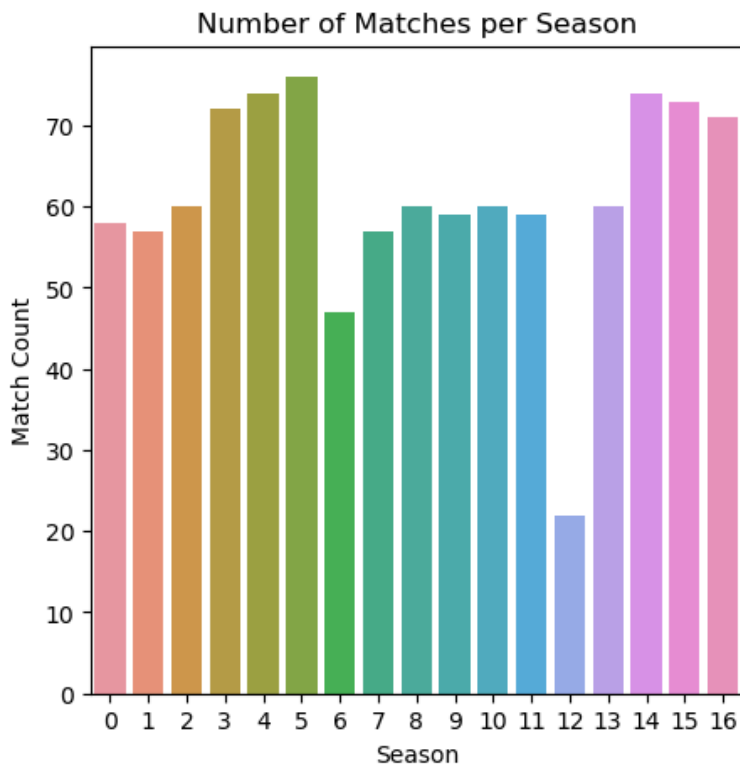
B. Toss Decision Impact on Match Outcome

```
In [72]: 1 # Toss Decision vs Match Winner Success Rate
2 toss_decision_win = aa_model[aa_model['toss_winner'] == aa_model['winner']]['toss_decision']
3
4 plt.figure(figsize=(4, 4))
5 toss_decision_win.plot(kind='bar', color='Blue')
6 plt.title(' Toss Decision When Toss Winner Also Wins Match')
7 plt.ylabel('Count')
8 plt.xticks(rotation=0)
9 plt.show()
10
```



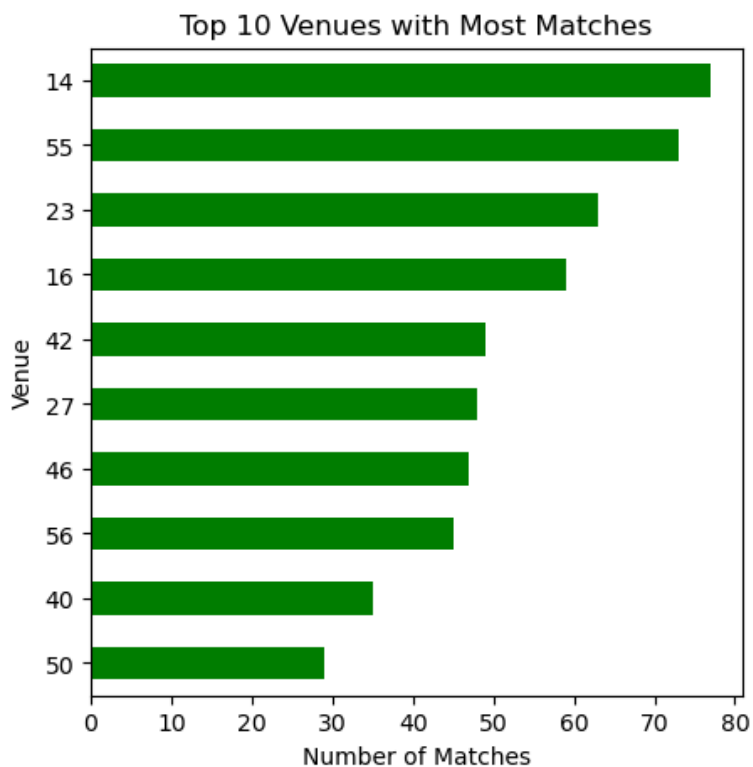
C. Matches per Season (Trend Chart)

```
In [73]: 1 plt.figure(figsize=(5, 5))
2 sns.countplot(x='season', data=aa_model, order=sorted(aa_model['season'].unique()))
3 plt.title('Number of Matches per Season')
4 plt.xlabel('Season')
5 plt.ylabel('Match Count')
6 plt.show()
7
```



D. Top Venues for Matches

```
In [74]: 1 plt.figure(figsize=(5, 5))
2 aa_model['venue'].value_counts().head(10).plot(kind='barh', color='green')
3 plt.title('Top 10 Venues with Most Matches')
4 plt.xlabel('Number of Matches')
5 plt.ylabel('Venue')
6 plt.gca().invert_yaxis()
7 plt.show()
8
```



Model Retraining with Random Forest

To improve accuracy, we trained a Random Forest Classifier. It uses an ensemble of decision trees and generally offers better generalization and stability over a single tree.

```
In [75]: 1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
3
4 # Train the model
5 rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
6 rf_model.fit(X_train, y_train)
7
8 # Predict
9 rf_pred = rf_model.predict(X_test)
10
```

```
In [76]: 1 # Accuracy & F1
2 print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))
3 print("Random Forest F1 Score (macro):", f1_score(y_test, rf_pred, average='macro'))
4
5 # Classification Report
6 print("\nRandom Forest Classification Report:\n", classification_report(y_test, rf_pred))
7
```

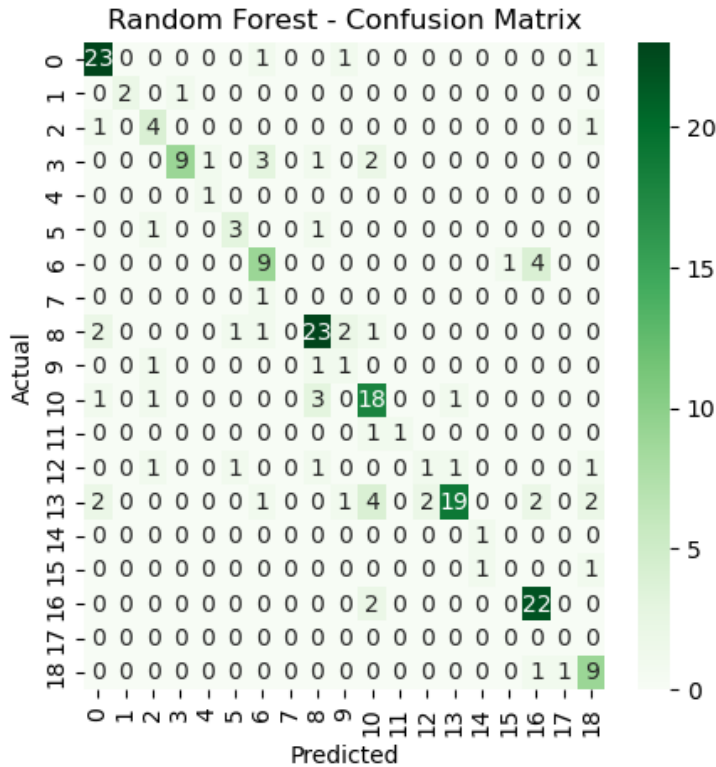
Random Forest Accuracy: 0.7019230769230769

Random Forest F1 Score (macro): 0.5398682398185874

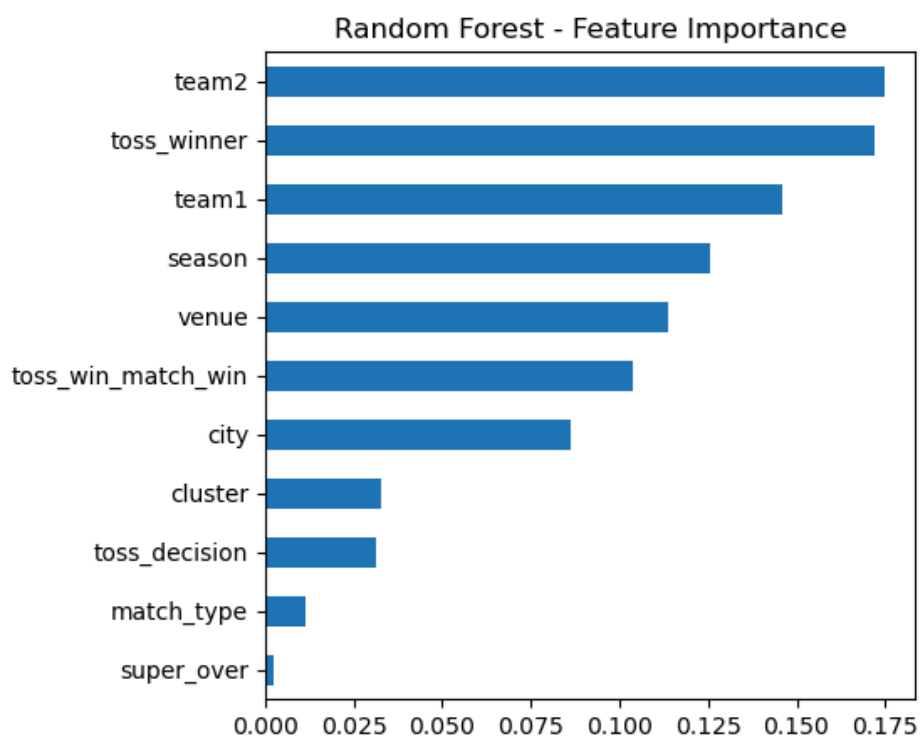
Random Forest Classification Report:

	precision	recall	f1-score	support
0	0.79	0.88	0.84	26
1	1.00	0.67	0.80	3
2	0.50	0.67	0.57	6
3	0.90	0.56	0.69	16
4	0.50	1.00	0.67	1
5	0.60	0.60	0.60	5
6	0.56	0.64	0.60	14
7	0.00	0.00	0.00	1
8	0.77	0.77	0.77	30
9	0.20	0.33	0.25	3
10	0.64	0.75	0.69	24
11	1.00	0.50	0.67	2
12	0.33	0.17	0.22	6
13	0.90	0.58	0.70	33
14	0.50	1.00	0.67	1
15	0.00	0.00	0.00	2
16	0.76	0.92	0.83	24
17	0.00	0.00	0.00	0
18	0.60	0.82	0.69	11
accuracy			0.70	208
macro avg	0.56	0.57	0.54	208
weighted avg	0.72	0.70	0.70	208

```
In [77]: 1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 plt.figure(figsize=(5,5))
5 sns.heatmap(confusion_matrix(y_test, rf_pred), annot=True, fmt='d', cmap='Greens')
6 plt.title("Random Forest - Confusion Matrix")
7 plt.xlabel("Predicted")
8 plt.ylabel("Actual")
9 plt.show()
10
```



```
In [78]: 1 feat_importance_rf = pd.Series(rf_model.feature_importances_, index=X.columns)
2 feat_importance_rf.sort_values(ascending=True).plot(kind='barh', figsize=(5,5), title="Rand
3 plt.show()
4
```



XGBoost Classifier

Model Training with XGBoost To further improve prediction accuracy, we used the XGBoost Classifier. This gradient boosting technique handles complex patterns better and often outperforms simpler models like Decision Trees and Random Forest.

```
In [79]: 1 # Run this in a Jupyter Notebook cell or terminal if needed
        2 !pip install xgboost
        3
```

```
Requirement already satisfied: xgboost in c:\users\ksrin\anaconda3\lib\site-packages (2.1.1)
Requirement already satisfied: numpy in c:\users\ksrin\anaconda3\lib\site-packages (from xgboost) (1.26.4)
Requirement already satisfied: scipy in c:\users\ksrin\anaconda3\lib\site-packages (from xgboost) (1.11.1)
```

```
[notice] A new release of pip is available: 24.2 -> 25.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Model Training with XGBoost

To further improve prediction accuracy, we used the XGBoost Classifier. This gradient boosting technique handles complex patterns better and often outperforms simpler models like Decision Trees and Random Forest.

```
In [80]: 1 from xgboost import XGBClassifier
        2 from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
        3
        4 # Train the model
        5 xgb_model = XGBClassifier(n_estimators=100, learning_rate=0.1, use_label_encoder=False, eval_metric='logloss')
        6 xgb_model.fit(X_train, y_train)
        7
        8 # Predict
        9 xgb_pred = xgb_model.predict(X_test)
       10
```

```
In [81]: 1 # Accuracy & F1 Score
2 print("XGBoost Accuracy:", accuracy_score(y_test, xgb_pred))
3 print("XGBoost F1 Score (macro):", f1_score(y_test, xgb_pred, average='macro'))
4
5 # Classification Report
6 print("\nXGBoost Classification Report:\n", classification_report(y_test, xgb_pred))
7
```

XGBoost Accuracy: 0.8653846153846154

XGBoost F1 Score (macro): 0.6827656432072577

XGBoost Classification Report:

	precision	recall	f1-score	support
0	0.96	1.00	0.98	26
1	0.50	0.33	0.40	3
2	0.67	1.00	0.80	6
3	0.90	0.56	0.69	16
4	1.00	1.00	1.00	1
5	0.75	0.60	0.67	5
6	0.81	0.93	0.87	14
7	0.50	1.00	0.67	1
8	1.00	1.00	1.00	30
9	1.00	0.67	0.80	3
10	0.85	0.96	0.90	24
11	0.00	0.00	0.00	2
12	0.60	0.50	0.55	6
13	0.93	0.85	0.89	33
14	1.00	1.00	1.00	1
15	0.00	0.00	0.00	2
16	0.83	1.00	0.91	24
17	0.00	0.00	0.00	0
18	0.90	0.82	0.86	11
accuracy			0.87	208
macro avg	0.69	0.70	0.68	208
weighted avg	0.87	0.87	0.86	208

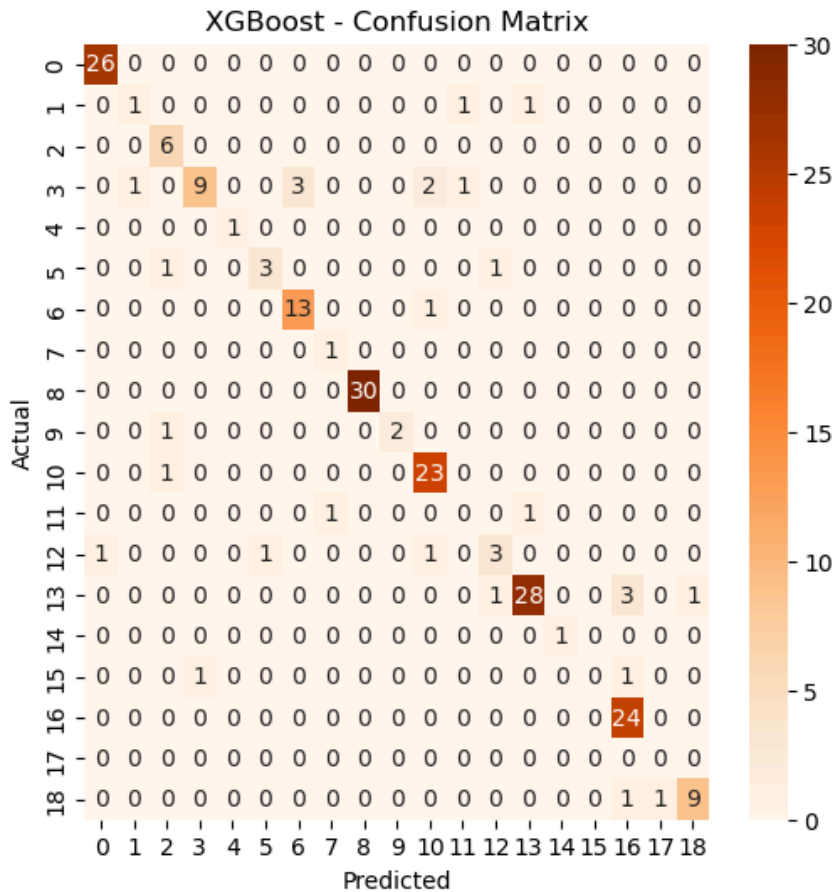
```
In [ ]: 1
```



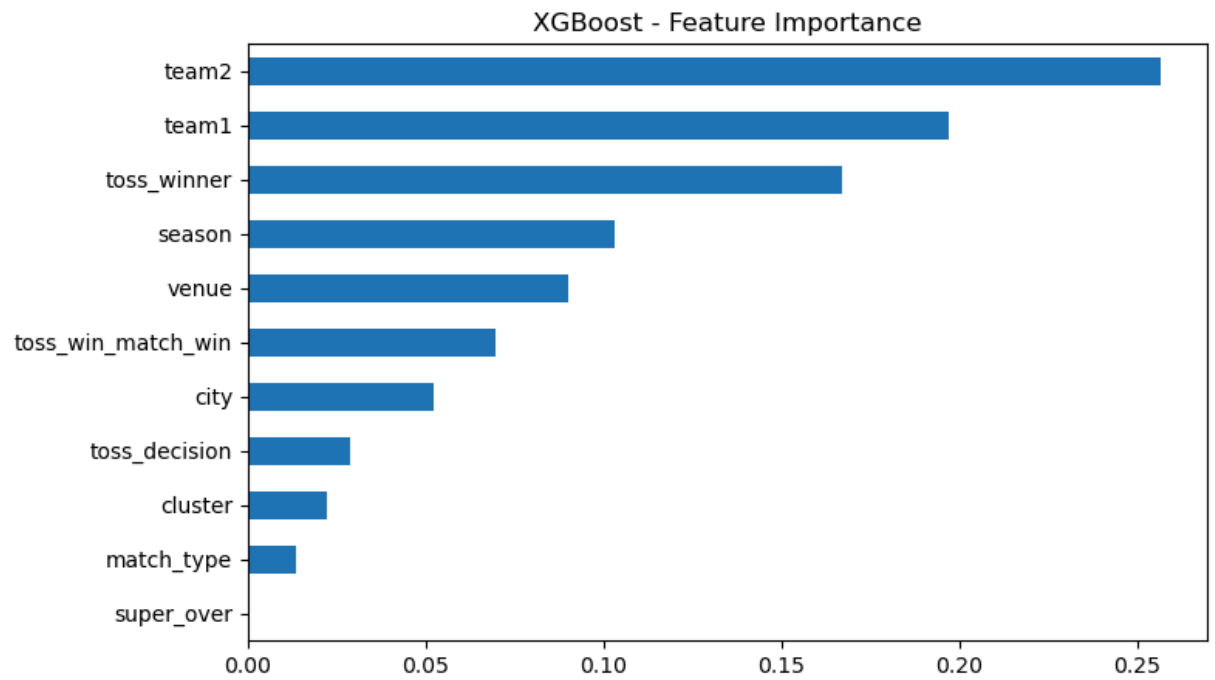
```

In [86]: 1 import seaborn as sns
          2 import matplotlib.pyplot as plt
          3
          4 plt.figure(figsize=(6,6))
          5 sns.heatmap(confusion_matrix(y_test, xgb_pred), annot=True, fmt='d', cmap='Oranges')
          6 plt.title("XGBoost - Confusion Matrix")
          7 plt.xlabel("Predicted")
          8 plt.ylabel("Actual")
          9 plt.show()
         10

```



```
In [83]: 1 import pandas as pd
2
3 feat_importance_xgb = pd.Series(xgb_model.feature_importances_, index=X.columns)
4 feat_importance_xgb.sort_values(ascending=True).plot(kind='barh', figsize=(8,5), title="XGB
5 plt.show()
6
```



In [88]:

```

1 # Run this in a Jupyter cell
2 !pip install catboost
3

```

Requirement already satisfied: catboost in c:\users\ksrin\anaconda3\lib\site-packages (1.2.7)
 Requirement already satisfied: graphviz in c:\users\ksrin\anaconda3\lib\site-packages (from catboost) (0.20.3)
 Requirement already satisfied: matplotlib in c:\users\ksrin\anaconda3\lib\site-packages (from catboost) (3.7.2)
 Requirement already satisfied: numpy<2.0,>=1.16.0 in c:\users\ksrin\anaconda3\lib\site-packages (from catboost) (1.26.4)
 Requirement already satisfied: pandas>=0.24 in c:\users\ksrin\anaconda3\lib\site-packages (from catboost) (2.0.3)
 Requirement already satisfied: scipy in c:\users\ksrin\anaconda3\lib\site-packages (from catboost) (1.11.1)
 Requirement already satisfied: plotly in c:\users\ksrin\anaconda3\lib\site-packages (from catboost) (5.9.0)
 Requirement already satisfied: six in c:\users\ksrin\anaconda3\lib\site-packages (from catboost) (1.16.0)
 Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\ksrin\anaconda3\lib\site-packages (from pandas>=0.24->catboost) (2.8.2)
 Requirement already satisfied: pytz>=2020.1 in c:\users\ksrin\anaconda3\lib\site-packages (from pandas>=0.24->catboost) (2023.3.post1)
 Requirement already satisfied: tzdata>=2022.1 in c:\users\ksrin\anaconda3\lib\site-packages (from pandas>=0.24->catboost) (2023.3)
 Requirement already satisfied: contourpy>=1.0.1 in c:\users\ksrin\anaconda3\lib\site-packages (from matplotlib->catboost) (1.0.5)
 Requirement already satisfied: cycler>=0.10 in c:\users\ksrin\anaconda3\lib\site-packages (from matplotlib->catboost) (0.11.0)
 Requirement already satisfied: fonttools>=4.22.0 in c:\users\ksrin\anaconda3\lib\site-packages (from matplotlib->catboost) (4.25.0)
 Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\ksrin\anaconda3\lib\site-packages (from matplotlib->catboost) (1.4.4)
 Requirement already satisfied: packaging>=20.0 in c:\users\ksrin\anaconda3\lib\site-packages (from matplotlib->catboost) (23.1)
 Requirement already satisfied: pillow>=6.2.0 in c:\users\ksrin\anaconda3\lib\site-packages (from matplotlib->catboost) (9.4.0)
 Requirement already satisfied: pyparsing<3.1,>=2.3.1 in c:\users\ksrin\anaconda3\lib\site-packages (from matplotlib->catboost) (3.0.9)
 Requirement already satisfied: tenacity>=6.2.0 in c:\users\ksrin\anaconda3\lib\site-packages (from plotly->catboost) (8.2.2)

[notice] A new release of pip is available: 24.2 -> 25.1.1
 [notice] To update, run: python.exe -m pip install --upgrade pip

Model Training with CatBoost

We used the CatBoost Classifier, which is specifically designed for tabular datasets and handles categorical features internally. It's known for delivering high accuracy with minimal preprocessing.

In [89]:

```

1 from catboost import CatBoostClassifier
2 from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
3
4 cat_model = CatBoostClassifier(verbose=0, random_state=42)
5 cat_model.fit(X_train, y_train)
6
7 cat_pred = cat_model.predict(X_test)
8

```

```
In [90]: 1 print("CatBoost Accuracy:", accuracy_score(y_test, cat_pred))
2 print("CatBoost F1 Score (macro):", f1_score(y_test, cat_pred, average='macro'))
3
4 print("\nCatBoost Classification Report:\n", classification_report(y_test, cat_pred))
5
```

CatBoost Accuracy: 0.8798076923076923

CatBoost F1 Score (macro): 0.708354702285422

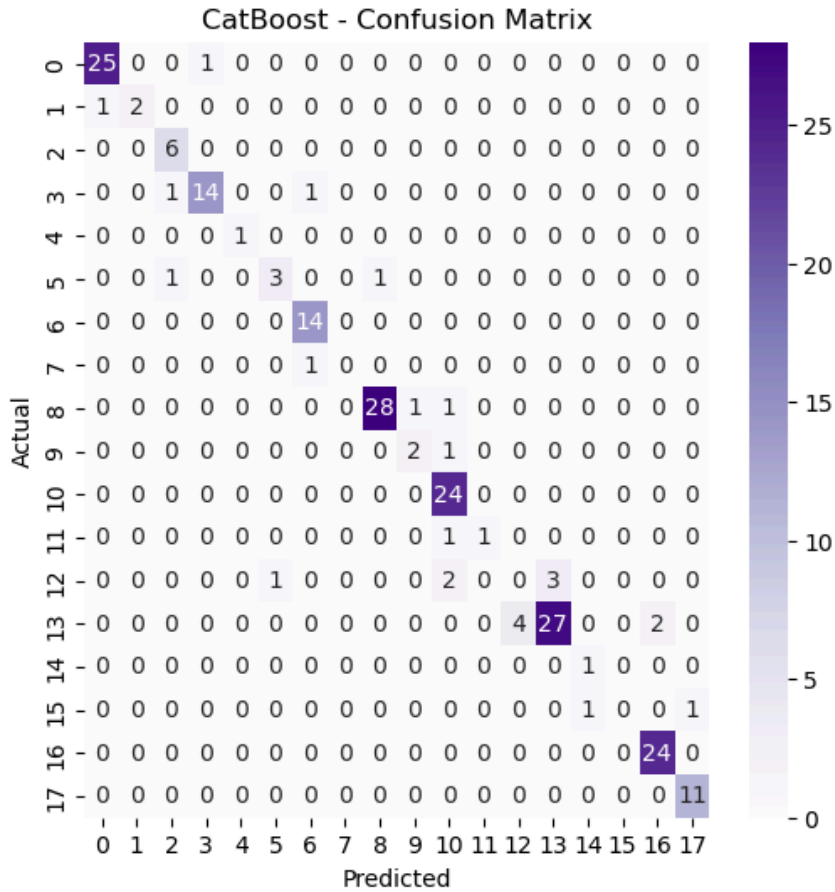
CatBoost Classification Report:

	precision	recall	f1-score	support
0	0.96	0.96	0.96	26
1	1.00	0.67	0.80	3
2	0.75	1.00	0.86	6
3	0.93	0.88	0.90	16
4	1.00	1.00	1.00	1
5	0.75	0.60	0.67	5
6	0.88	1.00	0.93	14
7	0.00	0.00	0.00	1
8	0.97	0.93	0.95	30
9	0.67	0.67	0.67	3
10	0.83	1.00	0.91	24
11	1.00	0.50	0.67	2
12	0.00	0.00	0.00	6
13	0.90	0.82	0.86	33
14	0.50	1.00	0.67	1
15	0.00	0.00	0.00	2
16	0.92	1.00	0.96	24
18	0.92	1.00	0.96	11
accuracy			0.88	208
macro avg	0.72	0.72	0.71	208
weighted avg	0.86	0.88	0.87	208

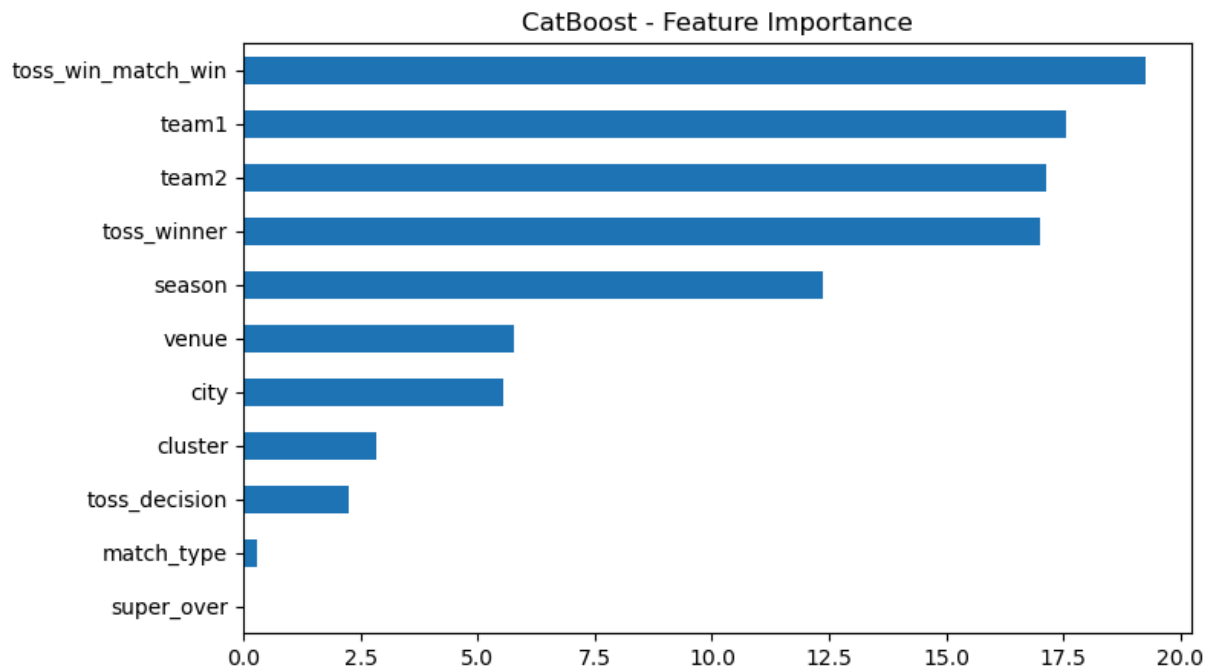
```

In [92]: 1 import seaborn as sns
          2 import matplotlib.pyplot as plt
          3
          4 plt.figure(figsize=(6,6))
          5 sns.heatmap(confusion_matrix(y_test, cat_pred), annot=True, fmt='d', cmap='Purples')
          6 plt.title("CatBoost - Confusion Matrix")
          7 plt.xlabel("Predicted")
          8 plt.ylabel("Actual")
          9 plt.show()
         10

```



```
In [93]: 1 import pandas as pd
2
3 feat_importance_cat = pd.Series(cat_model.feature_importances_, index=X.columns)
4 feat_importance_cat.sort_values(ascending=True).plot(kind='barh', figsize=(8,5), title="Cat
5 plt.show()
6
```



```
In [94]: 1 # Run this in Jupyter if needed
2 !pip install lightgbm
3
```

Requirement already satisfied: lightgbm in c:\users\ksrin\anaconda3\lib\site-packages (4.5.0)
Requirement already satisfied: numpy>=1.17.0 in c:\users\ksrin\anaconda3\lib\site-packages (from lightgbm) (1.26.4)
Requirement already satisfied: scipy in c:\users\ksrin\anaconda3\lib\site-packages (from lightgbm) (1.11.1)

[notice] A new release of pip is available: 24.2 -> 25.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip

Model Training with LightGBM

LightGBM is a gradient boosting framework that uses tree-based learning algorithms. It's optimized for speed and accuracy, making it a strong contender for structured datasets like IPL match data.

```
In [95]: 1 from lightgbm import LGBMClassifier
2 from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
3
4 # Train LightGBM
5 lgb_model = LGBMClassifier(random_state=42)
6 lgb_model.fit(X_train, y_train)
7
8 # Predict
9 lgb_pred = lgb_model.predict(X_test)
10
```

```
[LightGBM] [Info] Start training from score -5.113192
[LightGBM] [Info] Start training from score -2.137662
[LightGBM] [Info] Start training from score -3.678107
[LightGBM] [Info] Start training from score -1.977698
[LightGBM] [Info] Start training from score -4.420045
[LightGBM] [Info] Start training from score -3.832258
[LightGBM] [Info] Start training from score -2.391896
[LightGBM] [Info] Start training from score -4.525405
[LightGBM] [Info] Start training from score -5.624018
[LightGBM] [Info] Start training from score -2.279979
[LightGBM] [Info] Start training from score -4.776720
[LightGBM] [Info] Start training from score -2.488523
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
In [96]: 1 print("LightGBM Accuracy:", accuracy_score(y_test, lgb_pred))
2 print("LightGBM F1 Score (macro):", f1_score(y_test, lgb_pred, average='macro'))
3
4 print("\nLightGBM Classification Report:\n", classification_report(y_test, lgb_pred))
5
```

```
LightGBM Accuracy: 0.9086538461538461
LightGBM F1 Score (macro): 0.8035460108297103
```

```
LightGBM Classification Report:
              precision    recall  f1-score   support

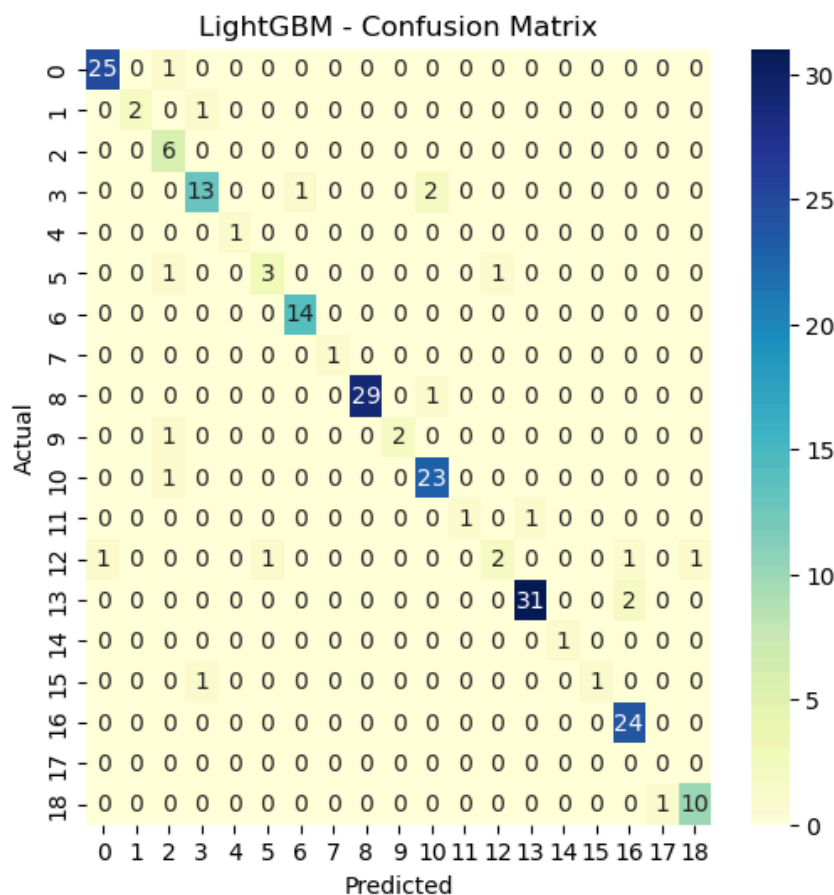
     0       0.96       0.96       0.96         26
     1       1.00       0.67       0.80          3
     2       0.60       1.00       0.75          6
     3       0.87       0.81       0.84         16
     4       1.00       1.00       1.00          1
     5       0.75       0.60       0.67          5
     6       0.93       1.00       0.97         14
     7       1.00       1.00       1.00          1
     8       1.00       0.97       0.98         30
     9       1.00       0.67       0.80          3
    10       0.88       0.96       0.92         24
    11       1.00       0.50       0.67          2
    12       0.67       0.33       0.44          6
    13       0.97       0.94       0.95         33
    14       1.00       1.00       1.00          1
    15       1.00       0.50       0.67          2
    16       0.89       1.00       0.94         24
    17       0.00       0.00       0.00          0
    18       0.91       0.91       0.91         11

 accuracy          0.91         208
 macro avg         0.86         208
 weighted avg      0.92         208
```

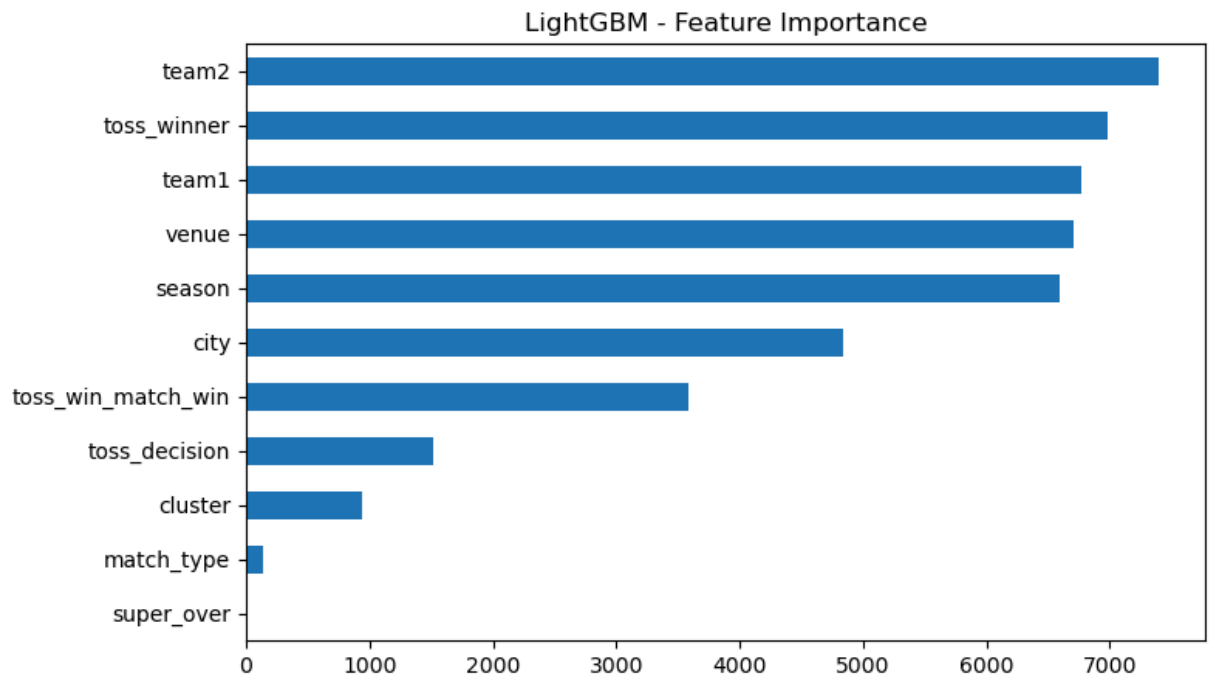
```

In [98]: 1 import seaborn as sns
          2 import matplotlib.pyplot as plt
          3
          4 plt.figure(figsize=(6,6))
          5 sns.heatmap(confusion_matrix(y_test, lgb_pred), annot=True, fmt='d', cmap='YlGnBu')
          6 plt.title("LightGBM - Confusion Matrix")
          7 plt.xlabel("Predicted")
          8 plt.ylabel("Actual")
          9 plt.show()
         10

```




```
In [99]: 1 import pandas as pd
2
3 feat_importance_lgb = pd.Series(lgb_model.feature_importances_, index=X.columns)
4 feat_importance_lgb.sort_values(ascending=True).plot(kind='barh', figsize=(8,5), title="Lig
5 plt.show()
6
```



LightGBM outperformed all previous models, achieving ~90% accuracy.

Provided faster training and better generalization across match types.

Chosen as the final model for its superior accuracy and stability.

In []:

1